

Programming Languages

Types & Static vs Dynamic Typing

1. Types of Programming Languages

Procedural

- Executes step-by-step instructions in sequence.
- Focuses on functions/procedures.
- **Examples:** C, Pascal, COBOL, Fortran
- **Use:** System programming, scripting, OS development

Object-Oriented (OOP)

- Organizes code around objects (data + behavior).
- Supports encapsulation, inheritance, polymorphism.
- **Examples:** Java, C++, Python, C#
- **Use:** Enterprise apps, GUI, game dev

Functional

- Treats computation as evaluation of mathematical functions.
- Avoids shared state and mutable data.
- **Examples:** Haskell, Erlang, Scala, Lisp
- **Use:** Data processing, concurrent systems

Scripting

- Interpreted; automates tasks and controls other programs.
- **Examples:** Python, JavaScript, Bash, Ruby
- **Use:** Web dev, automation, DevOps

Logic / Declarative

- Describes WHAT to compute, not HOW.
- **Examples:** Prolog, SQL, Datalog
- **Use:** AI/Prolog, database queries

Assembly / Low-Level

- Closest to machine code; hardware-specific.
- **Examples:** x86 Assembly, ARM Assembly
- **Use:** Embedded systems, OS kernels, drivers

Markup / Query

- Defines structure or queries data — not general-purpose.
- **Examples:** HTML, XML, SQL, CSS

- **Use:** Web pages, data storage, styling

2. Compiled vs Interpreted vs Hybrid

Type	How it works	Examples	Speed
Compiled	Entire code → machine code before run	C, C++, Rust, Go	Fast
Interpreted	Line-by-line execution at runtime	Python, Ruby, Bash	Slower
Hybrid (JIT)	Bytecode compiled, JIT-executed at runtime	Java, C#, Kotlin	Middle ground

3. Static vs Dynamic Typing

Static Typing

Variable types are known and checked at **compile time**.

Type check happens at	Compile time
Type declaration	Mandatory (or inferred by compiler)
Errors caught	Before program runs
Performance	Generally faster (compiler optimizes)
Flexibility	Less flexible
Examples	Java, C, C++, Go, Kotlin, Swift, Rust

Java Example (Static):

```
int age = 25;           // type declared explicitly
String name = "Yuktha";
age = "hello";          // ERROR at compile time!
```

Dynamic Typing

Variable types are resolved at **runtime**. No need to declare types.

Type check happens at	Runtime
Type declaration	Not required — inferred automatically
Errors caught	Only when that line executes
Performance	Slightly slower due to runtime checks
Flexibility	Highly flexible
Examples	Python, JavaScript, Ruby, PHP, Bash

Python Example (Dynamic):

```
age = 25                # no type needed
age = "twenty"          # OK! type changed at runtime
print(age + 1)          # ERROR only when this line runs
```

4. Strongly vs Weakly Typed

Feature	Strongly Typed	Weakly Typed
Implicit conversion	Not allowed	Allowed automatically
Type errors	Caught early (explicit cast needed)	May silently convert
Examples	Python, Java, Go, Ruby	C, JavaScript, PHP
Example behaviour	"5" + 5 → TypeError	"5" + 5 → "55" or 10

5. Quick Reference: Language Classification

Language	Paradigm	Typing	Execution
Java	OOP	Static + Strong	Hybrid (JVM)
Python	Multi-paradigm	Dynamic + Strong	Interpreted
C	Procedural	Static + Weak	Compiled
C++	OOP + Procedural	Static + Weak	Compiled
JavaScript	Multi-paradigm	Dynamic + Weak	Interpreted/JIT
Go	Procedural + OOP	Static + Strong	Compiled
Kotlin	OOP + Functional	Static + Strong	Hybrid (JVM)
Ruby	OOP	Dynamic + Strong	Interpreted
Haskell	Functional	Static + Strong	Compiled
SQL	Declarative	Static	Interpreted

Key Takeaway: Java = Static + Strong + Hybrid. Python = Dynamic + Strong + Interpreted. Understanding typing helps you debug faster and choose the right language for the job.